

Algoritmusok vezérlési szerkezetei és megvalósításuk C programozási nyelven. A szekvenciális, iterációs, elágazásos, és az eljárás vezérlés.

ALGORITMUSOK VEZÉRLÉSI SZERKEZETEI ÉS MEGVALÓSÍTÁSUK C PROGRAMOZÁSI NYELVEN

Algoritmusok

- egy adott típusú feladat megoldására vonatkozó pontos előírás, amely meghatározza, hogy a kezdeti adatokon milyen műveleteket, milyen sorrendben kell elvégezni
- tulajdonságai
 - **meghatározott** (ugyanazon inputra mindig ugyanaz az eredmény)
 - **széleskörű** (egy feladatosztályra vonatkozik, nem egy feladatra)
 - **véges** (véges lépésben véget ér)

Algoritmusok vezérlése

- az előírás, amely az algoritmus minden lépéséhez kijelöli, hogy annak végrehajtása után melyik következő lépéssel kell folytatni vagy épp befejezni magát a folyamatot
- az algoritmusok legfontosabb komponense a vezérlés, ez határozza meg a dinamikus működését
- távol áll a megvalósítástól
- leírás módjai
 - természetes nyelvi leírás
 - szöveges leírás, mondatokba foglalva
 - pszeudokód
 - programozási nyelvhez hasonló strukturált nyelv, csak szabadabb
 - folyamat ábra
 - grafikus, kevésbé strukturált gráf ami a végrehajtásra, a folyamatra koncentrálnak
 - szerkezeti ábra
 - grafikus, strukturált leírása az algoritmus felépítésének, amely leírja a működési folyamatot is
- típusai
 - **szekvenciális vezérlés (soros végrehajtás)**
 - **iterációs vezérlés (ismétléses vagy ciklusos):** for, while, do while
 - **elágazásos vezérlés (szelekció):** if-else if-else, switch
 - **eljárás vezérlés (modularitás):** függvények

Szekvenciális vezérlés

- az algoritmusok alapértelmezett működési módja, ahol az utasítások egymás után, abban a sorrendbe hajtódnak végre, ahogyan azok a forráskódban vannak írva
 - minden utasítás csak egyszer hajtódik végre
 - nincs ugrás és nincs feltételvizsgálat sem
- C nyelvben
 - C-ben minden egyszerű utasítást pontosvessző (;) zár le
 - a kapcsos zárójelek { } közé zárt utasítássorozatot a C nyelv egyetlen utasításnak (blokknak) tekinti
 - a blokk vagy az összetett utasítás **szintaktikailag egyetlen utasításnak felel meg** és **saját hatókört** (scope) **definiál** azaz egy adott blokkon belül deklarált változók csak azon blokkon belül érhetők el

```
#include <stdio.h>

int main() {
    int a = 5;
    int b = 10;
    int sum = a + b;
    printf("Sum: %d\n", sum);
    return 0;
}
```

Elágazásos vezérlés (szelekció)

- lehetővé teszi, hogy véges sok rögzített művelet közül egy vagy több feltétel alapján válasszuk ki, hogy melyik művelet (ág) kerüljön végrehajtásra
- típusai:
 - egyszerű szelekció (if)
 - egyetlen feltétel dönt arról, hogy végrehajtunk-e egy műveletet vagy sem
 - kétirányú szelekció (if-else)
 - egy feltétel van, ha az igaz akkor az "A" művelet hajtódik végre, ha pedig hamis a feltétel akkor a "B" művelet hajtódik végre
 - többszörös szelekció (if-else if)
 - több feltételünk és műveletünk van
 - a feltételeket sorba vizsgáljuk és az első igaz feltételhez tartozó művelet fut le
 - ha mind hamis volt és nincs *else* ág akkor nem fut le egyik se, ha volt *else* ág akkor annak a művelete hajtódik végre
 - esetkiválasztós szelekció (switch)
 - egy kifejezés (szelektor) értéke alapján választunk a lehetséges ágak (case-ek) közül
 - if-else if-el is megvalósítható, de switch-el sokkal átláthatóbb és könnyebben olvasható kód jön létre, főleg ha nagyon sok ág van

- **if** C nyelvben
 - az **if** szerkezetben logikai operátorok segítségével több feltételt tudunk kombinálni
 - fontos, hogy C-ben a feltétel lehet szám is
 - 0 érték jelenti a false-t
 - minden nem 0 érték pedig a true-t
 - leggyakorabbi logikai operátorok az ÉS (&&), illetve a VAGY (||)
 - ÉS akkor ad igazat, ha mindkét operandus true
 - VAGY akkor ad igazat, ha legalább az egyik operandus true
 - **rövidzár** (short-circuit)
 - lényege, hogy amennyiben első feltétel után eldől a kiemenet akkor a többi feltételt ki sem értékeli
 - példák
 - **a == 0 && b > 1**
 esetén ha *a* nem nulla akkor tudjuk, hogy az ÉS false-t fog adni ezért a *b > 1* -t ki sem értékeljük feleslegesen
 - de ha *a* = 0 akkor természetesen ki kell értékelni a *b > 1*-et is
 - **a == 0 || b > 1**
 esetén ha *a* = 0 akkor tudjuk, hogy a VAGY true-t fog adni ezért a második feltételt ki sem értékeli
 - ez védelmi funkció is, mivel hibákat is meg tudunk akadályozni
 - **if (mutato != NULL && mutato->ertekek > 0)**
 esetén ha a *mutato* = *NULL* akkor a rövidzárnak köszönhetően a második feltétel nem kerül kiértékelésre, ez azért jó mert ha kiértékelésre kerülne hibát dobna
 - de hibákat is okozhat
 - **if (a == 5 || ++b > 10)**
 esetén ha *a* = 5, akkor a VAGY miatt a kifejezés igaz lesz és a jobb oldalon lévő *++b* művelet nem hajtodik végre, így *b* értéke nem változik, ami logikai hibához vezethet, ha a programozó számított a *b* érték változására
 - **if** és **if-else** szerkezet C-ben
 - **if(feltétel) { /*igaz ág*/ }**

```

if (feltétel) {
    /* igaz ág */
} else {
    /* hamis ág */
}

```
 - **else if** nincs C-ben, de egymásba ágyazott **if-else**-ekkel megoldható
 - **if(feltétel) {**
 } else {
 if(feltétel2) { }
 }

- **switch** a C nyelvben (esetkiválasztás)
 - akkor használjuk, ha egy egész típusú kifejezés (int, char és egyéb long, stb.) értéke szerint kell sok irányba ágazni
 - **break** utasítás fontos az ágakban (case), mivel ha hiányzik a vezérlés átsorog a következő ágra (bár néha ki lehet használni)

```
switch (nap_szama) {
    case 1: printf("Hétfő"); break;
    case 2: printf("Kedd"); break;
    default: printf("Egyéb nap"); break;
}
```

- például ha az ábrán lévő *switch*-ben a *nap_szama* = 2 és a kiírás utána a *case 2*-be nem lenne *break* akkor a “Kedd” mellett kiíródna még az “Egyéb nap” is

Iterációs vezérlés (ismétléses vezérlés)

- lehetővé teszi, hogy egy adott utasítássorozat többször ismétlődjön, addig amíg egy bizonyos feltétel teljesül
- ciklusok általános felépítése
 - **Ciklusmag:** az a művelet vagy szekvencia, amelyet többször végrehajtunk
 - **Ciklusfeltétel:** egy logikai kifejezés, amely meghatározza, hogy hányszor vagy meddig fusson a ciklus
 - **Vezérlő változó (ciklusváltozó):** gyakran használt változó, amelynek értéke minden lépésben változik, és szerepel a feltételben (pl. ciklusváltozó: *i*)
- C nyelvben van többféle iterációs szerkezet
 - **while** (előtesztelő ciklus)
 - mindig először ellenőrzi a feltételt, majd lefut a ciklusmag, addig ismétlődik, amíg a feltétel igaz marad
 - ha kezdetkor hamis a feltétel le se fut a ciklusmag
 - **C megvalósítás:** while (feltétel) { műveletek; }
 - akkor használjuk, ha nem tudjuk előre a lépésszámot és fennáll a lehetősége, hogy egyáltalán nem is kell végrehajtani a műveletet
 - pl: fájl végig olvasása (pl. üres fájl esetén 1x se kell lefutnia)
 - speciális változata a **hurok**
 - ilyenkor a while ciklus végtelen (feltétele mindig true) és egy if szerkezetet használva, bizonyos feltételek teljesülése esetén megállítjuk a ciklust manuálisan (break)
 - while(true) { if(feltétel) break; }
 - **do-while** (háttesztelő ciklus)
 - a ciklus először mindenképp végrehajtódik, majd utána ellenőrzi a feltételt és addig fut még a feltétel igaz
 - tehát ha kezdetkor hamis is a feltétel egy alkalommal garantáltan le fut a ciklusmag
 - **C megvalósítás:** do { műveletek; } while (feltétel);
 - itt a végén kötelező a pontosvessző!
 - példa: felhasználótól való adatbekérés validálással
 - addig kérjük be a számot, amíg nem felel meg a feltételeknek, de egyszer le kell kérni úgylis

- **for** (számláló ciklus)
 - speciális előtesztelő ciklus
 - egy előre meghatározott számú iteráció végrehajtására használjuk
 - minden for ciklus átírható egy vele ekvivalens while ciklusra
 - **C megvalósítása:** for (inicializálás; feltétel; léptetés) { ... }
 - for(int i = 0; i < 10; i++)
 - működése
 1. inicializálás (egyszer fut le csak)
 2. feltétel ellenőrzése (ha hamis kilép)
 3. ciklusmag végrehajtása
 4. léptetés végrehajtása
 5. ugrás a 2. lépéshez
- **foreach ciklus** (diszkrét (gyűjtemény) alapú ciklus)
 - listák és tömbök teljes bejárására alkalmazzák
 - C nyelvben nincs beépített foreach szerkezet, de for ciklussal megoldható
 - for (i = 0; i < n; i++) { a[i]... }
 - *n* az *a* tömb hossza
- **Ciklus vezérlő utasítások**
 - használatuk kerülendő a strukturált programozás elvei alapján
 - mert több kilépési pontot hoznak létre
 - **break:** azonnal megszakítja a legbelsőbb ciklus (esetleg switch) futását és a vezérlés a ciklus utáni első utasítására ugrik
 - **continue:** megszakítja az aktuális iterációt és a következő iterációra ugrik és újra ellenőrzi a feltételt
 - **goto:** segítségével a program a kód bizonyos részeire tud ugrani (akár a saját blokkját is újra hívhatja visszafelé)

```
#include <stdio.h>

int main() {
    int n = 0;

    // If the number is zero, jump to
    // jump_here label
    if (n == 0)
        goto jump_here;

    // This will be skipped
    printf("You entered: %d\n", n);

jump_here:
    printf("Exiting the program.\n");
    return 0;
}
```

```
#include <stdio.h>

int main(){
    int n = 1;

    // Label here
label:
    printf("%d ", n);
    n++;
    if (n <= 10)

        // jumb back to the label
        goto label;
    return 0;
}
```

Eljárás vezérlés (modularitás)

- **modularitás** lényege, hogy a bonyolult problémát részproblémákra bontjuk és ezeket külön egységekként (**függvényekként**) valósítjuk meg
- eljárás (procedure) = nem ad vissza semmit | függvény = valamit ad vissza
- előnye a **kód újrafelhasználhatósága**
- **absztrakció**
 - a hívó félnek csak azt kell tudnia *mit csinál* a függvény (bemenet és kimenet), de azt nem, hogy *hogyan* (fekete doboz elv)
- **Függvények C-ben**
 - minden alprogram egy függvény C-ben
 - nincs külön “eljárás” kulcsszó, mint pl js esetén a function
 - ha nincs egy függvénynek visszatérési értéke akkor **void** típusúnak deklaráljuk
 - **deklaráció (prototípus)**
 - megadja a függvény visszatérési típusát, a nevét és a paramétereinek típusát és a nevüket
 - pl: int osszead(int a, int b);
 - **definíció**
 - a függvény tényleges törzse itt található meg a műveletek
 - **Vezérlés átadása**
 - amikor a program egy függvény híváshoz ér akkor:
 1. a vezérlés átadódik a hívott függvénynek
 2. az aktuális paraméterek értéke átadódik a formális paramétereknek
 3. lefut a függvény
 4. a vezérlés visszatér a hívás helye utáni következő utasításra
 - opcionálisan egy visszatérési értékkel
 - a gép a verem (stack) memóriaterületet használja a visszatérési címek és a lokális változók tárolására (ez teszi a rekurziót lehetővé)
 - mikor visszatér a vezérlés a függvény hívójához akkor a stack felszabadul
 - C nyelvben a vezérlés átadása alapértelmezés szerint **érték szerinti paraméterátadással** történik
 - azaz a függvény egy másolatot kap az adatokról, ezzel az eredeti változót nem tudja módosítani (kivéve, ha pointert használunk)
 - **Lokális változók**
 - egy függvényen vagy blokkon belül deklarált változó, csak az adott függvényen vagy blokkon belül érhető el, a függvény végén megszűnik
 - **Globális változók**
 - függvényeken kívül deklarált változók, ezeket minden függvény látja és módosíthatja is
 - használatuk átláthatatlanná teszi a programot, mert bármelyik függvény váratlanul módosíthatja az értéküket

Egyéb C-s dolgok

- **Prefix operátor**

- az operátor a változó értékének növelése vagy csökkentése a használat előtt történik meg

```
int i = 0;  
printf("Érték: %d", ++i);
```

- kimenet: "Érték: 1"

- **Postfix operátor**

- az operátor a változó értékének növelése vagy csökkentése után történik meg

```
int i = 0;  
printf("Érték: %d", i++);
```

- kimenet: "Érték: 0"

```
a = 5  
++a;      // a becomes 6  
a++;      // a becomes 7  
--a;      // a becomes 6  
a--;      // a becomes 5
```